

第二讲(续) K-近邻适用性引申的问题

---特征选择

主要内容

- 由K-近邻适用性引申的问题：特征选择
 - 过滤式策略下的特征选择
 - 包裹式策略下的特征选择
 - 嵌入式策略下的特征选择(后续讲)
- 以图像数据案例为例讨论
 - 图像数据的特点
 - 图像数据的组织，等

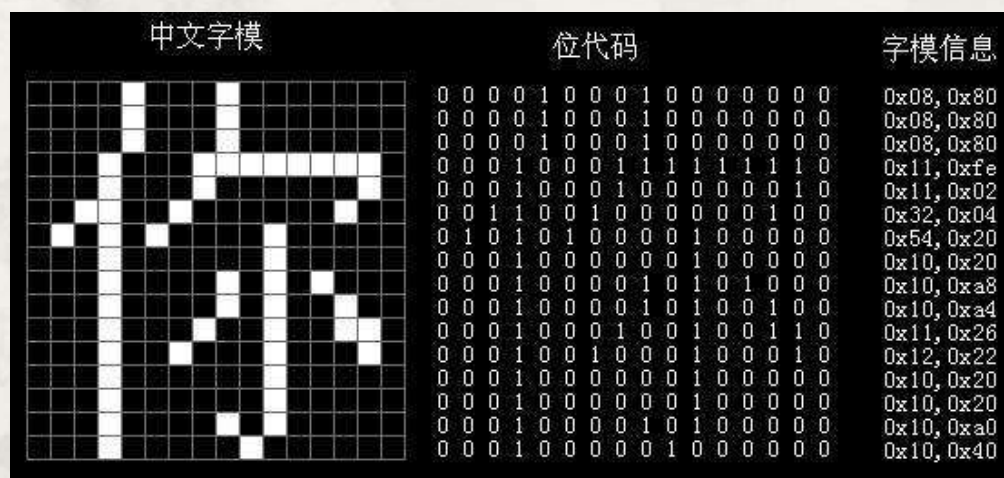
- 案例四：脸部表情的分类预测

- 对人脸图像数据集（文件名：脸部表情.txt），基于K-近邻法及特征选择策略，对脸部表情进行分类预测

- 表情代码：0：生气；1：厌恶；2：害怕；3：高兴；4：悲伤；5：惊讶，6：平静



- 案例四的数据：非结构化数据
 - 非结构化数据可简单理解为没有结构的数据
 - 典型的非结构化数据：文本、图像、音频和视频数据
 - 处理非结构化数据，需首先将非结构化数据进行向量化和结构化处理
 - 图像数据的向量化
 - 图像数据一般针对行列点阵，采用类似于计算机汉字字形码点阵的设计思路



- 图像的每个单元格用整数记录，整数大小表征对应单元格上图像灰度值的大小
 - 取值范围：0-255，0对应最大灰度即为黑实，255对最小灰度即为白色
- 本例中，每张脸部图像将用一组共计 $48 \times 48 = 2304$ 个有序的整型数表征
 - 称此为图像数据的向量化过程

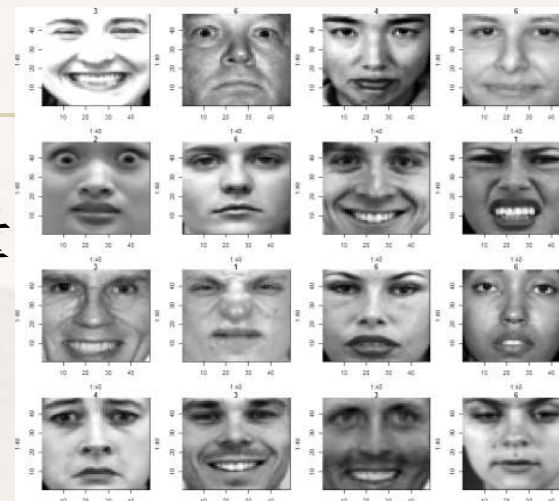
• 图像数据的结构化

• 展平方式：将 N 幅图像展平为结构化数据的组织形式

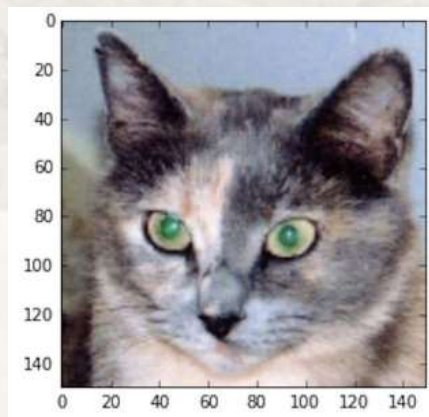
- 一行对应一幅图像，共 N 行对应 N 幅图像
- 本例中每行有2304列，依次为各单元格的灰度值变量
- 一般适合于经典机器学习算法

• 二维矩阵方式：

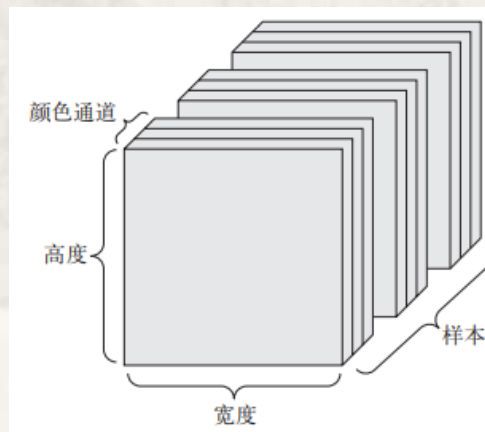
- 一幅脸部图像对应一个 48×48 的矩阵：2D张量
 - N 幅图像分别对应 N 个矩阵
 - 通常更适合于深度学习算法
- 彩色图像的3维:高度、宽度和颜色(RGB):单张图像可用3D张量表示(多张用4D表示)



48*48点阵黑白图像



150*150点阵彩色图像



- 若每张图像对应黑白视频的一个帧，一段黑白视频数据也可用3D张量表示

• 案例四的数据：组织结构和可视化



```
1 data=pd.read_csv('脸部表情.txt',header=0) #第2列为像素数据
2 data.head(5)
```

	Emotion	Pixels
0	3	221 240 251 254 255 255 255 255 255 255 255 25...
1	6	100 107 108 104 103 113 117 115 120 130 138 14...
2	4	35 50 56 57 63 76 74 79 85 86 105 133 145 152 ...
3	6	119 124 129 135 136 140 142 149 159 156 163 16...
4	2	160 173 186 194 188 185 175 162 153 143 135 12...

```
1 X = []
2 for i in np.arange(len(data)):
3     Xstr = data.loc[i,'Pixels'].split(' ')
4     X.append([int(x) for x in Xstr ])
5 print(np.shape(X))
6 Y = data['Emotion']
7
8 from sklearn import preprocessing (4178, 2304)
9 X = preprocessing.scale(X,axis=1)
```

- 0-255, 0黑实, 255白色
- 标准化处理后画图
- $48 \times 48 = 2304$ 维度较高

• 问：

- 若某些变量对预测作用不大，保留它可能出现的问题？
 - 复杂度，计算效率，参数求解和样本量
 - 特征选择和特征提取
- 案例四中哪些变量对表情分类的作用可能不大？

```
7 np.random.seed(1)
8 ids = np.random.choice(len(X),16)
9 plt.figure(figsize=(8,8))
10 for i,item in enumerate(ids):
11     img = X[item].reshape((48,48))
12     plt.subplot(4,4,i+1)
13     plt.imshow(img,cmap=cm.gray)
14 plt.show()
```

特征选择概述

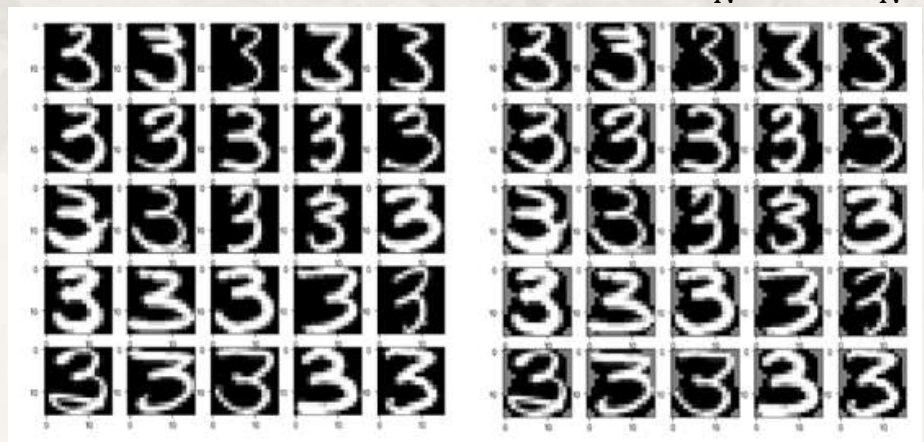
- 特征选择的角度
 - 第一，考察变量取值的差异程度
 - 第二，考察输入变量与输出变量的相关性
 - 第三，考察输入变量对模型误差的影响
- 具体策略：
 - 第一，**过滤式** (Filter) 策略
 - 特征选择与预测建模“分而治之”
 - 第二，**包裹式** (wrapper) 策略
 - 将特征选择“包裹”到一个指定的预测模型中，通过预测模型评价变量重要性，并完成变量筛选
 - 第三，**嵌入式** (embedding) 策略（以后讲）
 - 即特征选择“嵌入”到整个预测建模中，与预测建模“融为一体”

过滤式策略下的特征选择

- 考察变量取值的差异程度：**低方差过滤法** (Low Variance Filter)

- 若变量的取值差异很小，该变量不会对预测模型产生影响

- 对数值型变量计算方差
- 对分类型变量计算：
 - 类别 $k(k = 1, 2, \dots, K)$ 的样本量 N_k 与样本量 N 之比 $\frac{N_k}{N}$, ($0 < \frac{N_k}{N} \leq 1$)
 - 类别数 K 与样本量 N 之比 $\frac{K}{N}$, ($0 < \frac{K}{N} \leq 1$)



- 标准化的灰度值数据

```
1 data = pd.read_table('邮政编码数据.txt', sep=' ', header=None)
2 tmp = data.loc[data[0]==3]
3 X = tmp.iloc[:, 1:-1]
4 np.random.seed(1)
5 ids = np.random.choice(len(X), 25)
6 plt.figure(figsize=(8, 8))
7 for i, item in enumerate(ids):
8     img = np.array(X.iloc[item, :]).reshape((16, 16))
9     plt.subplot(5, 5, i+1)
10    plt.imshow(img, cmap=cm.gray)
```

Chapter10-2.ipynb

```
1 selector = VarianceThreshold(threshold=0.05)
2 selector.fit(X)
3 print("剩余变量个数: %d"%len(selector.get_support(True)))
4 X = selector.inverse_transform(selector.transform(X))
5 plt.figure(figsize=(8, 8))
6 for i, item in enumerate(ids):
7     img = np.array(X[item, :]).reshape((16, 16))
8     plt.subplot(5, 5, i+1)
9     plt.imshow(img, cmap=cm.gray)
```

剩余变量个数：221

过滤式策略下的特征选择

- 考察变量取值的差异程度：**低方差过滤法** (Low Variance Filter)
 - 案例四：利用低方差过滤后的脸部图像数据



- 问：其中比较关键的问题是？
 - 方差阈值的确定

• 过滤式策略下的特征选择：高相关过滤法

- 高相关过滤法：单个数值型输入变量 X 与分类型输出变量 y 相关性：方差分析

• 采用假设检验：

- 步骤1：原假 H_0 为： $\mu_1 = \mu_2 = \dots = \mu_k$ ，备则假设 H_1 为： $\mu_1 \neq \mu_2 \neq \dots \neq \mu_k$
- 步骤2：

- 计算原假设成立下，得到当前样本的特征或更极端特征的概率：概率-P值
- 概率-P值很小且小于显著性水平 α ，依据小概率原理，推翻原假设接受备择假设

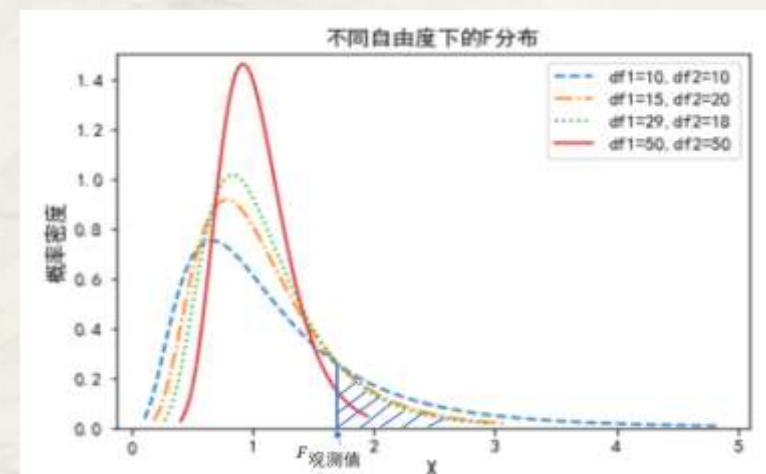
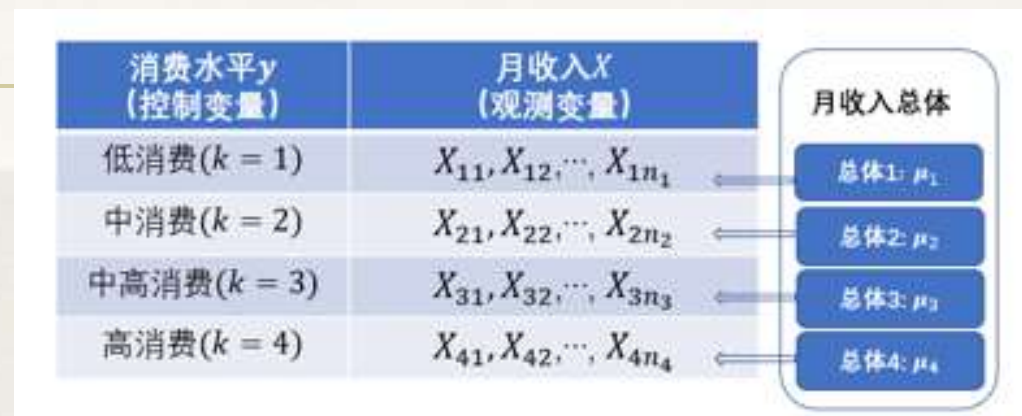
• 问：如何计算概率-P值？

$$SST = SSA + SSE$$

$$SST = \sum_{k=1}^K \sum_{i=1}^{N_k} (X_{ki} - \bar{X})^2 \quad SSA = \sum_{k=1}^K N_k (\bar{X}_k - \bar{X})^2 \quad SSE = \sum_{k=1}^K \sum_{i=1}^{N_k} (X_{ki} - \bar{X}_k)^2$$

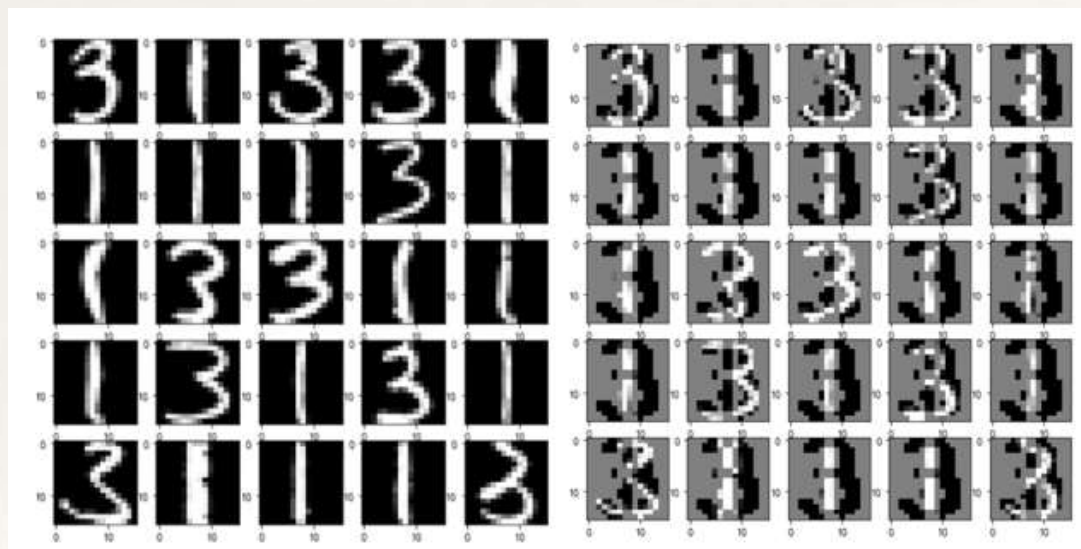
$$F_{\text{观测值}} = \frac{SSA/(K-1)}{SSE/(N-K)} = \frac{MSA}{MSE} \sim F(K-1, N-K)$$

概率-P值： $P(F \geq F_{\text{观测值}})$



- 过滤式策略下的特征选择

- 高相关过滤法：单个数值型输入变量 X 与分类型输出变量 y 相关性：方差分析



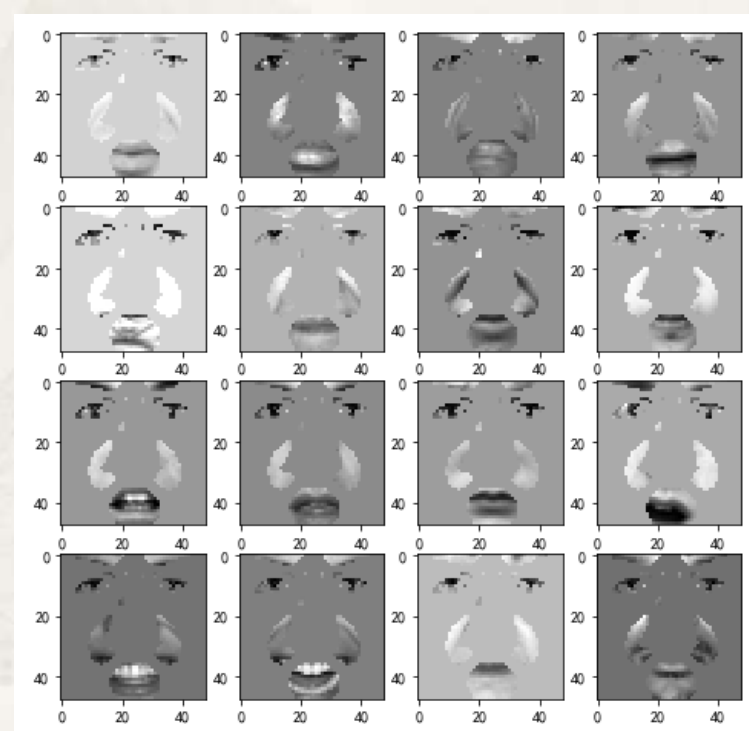
Chapter10-2.ipynb

```
1 tmp = data.loc[(data[0]==1) | (data[0]==3)]
2 X = tmp.iloc[:,1:-1]
3 Y = tmp.iloc[:,0].astype(str)
4 np.random.seed(1)
5 ids = np.random.choice(len(X), 25)
6 plt.figure(figsize=(8,8))
7 for i,item in enumerate(ids):
8     img = np.array(X.iloc[item,]).reshape((16,16))
9     plt.subplot(5,5,i+1)
10    plt.imshow(img, cmap=cm.gray)
11 plt.show()
12 selector = SelectKBest(score_func=f_classif, k=100)
13 selector.fit(X, Y)
14 print("变量重要性评分:", selector.scores_[0:5])
15 print("变量的概率P-值:", selector.pvalues_[0:5])
16 X = selector.inverse_transform(selector.transform(X))
17 plt.figure(figsize=(8,8))
18 for i,item in enumerate(ids):
19     img = np.array(X[item,]).reshape((16,16))
20     plt.subplot(5,5,i+1)
21     plt.imshow(img, cmap=cm.gray)
```

变量重要性评分: [5.10898495 21.93357405 82.3841514 282.84509937 719.07806468]
变量的概率P-值: [2.39312394e-002 3.05218712e-006 3.08578340e-019 9.82205018e-059
6.44143983e-132]

过滤式策略下的特征选择

- 考察变量取值的差异程度：**高相关过滤法**
 - 案例四：针对低方差过滤后再进行高相关过滤后的脸部图像数据



包裹式策略下的特征选择

- 包裹式策略下的特征选择
 - 将特征选择“包裹”到一个指定的预测模型中
 - 借助预测模型，依据变量对预测模型损失函数(或目标函数)影响的大小，给出变量重要性的打分
 - 在全部变量 $X_1, X_2, \dots, X_p$ 中，若变量 X_i 可以最大程度地减少损失函数值，则变量 X_i 最重要。反之，变量 X_i 最不重要
 - 结果：给出具有包含关系的输入变量子集 $F_1 \subset F_2 \subset \dots \subset F$
- 包裹式策略的基本思路
 - 反复迭代：每次迭代给出剔除当前最不重要变量后的特征子集

包裹式策略下的特征选择

- 递归式特征剔除法（Recursive Feature Elimination, RFE）：从计算效率角度，可直接依据预测模型中变量 X_i 的系数，判断变量的重要性
 - 例：回归模型
$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$
 - 设当前变量集 $\mathbf{S} = \mathbf{F} = \{X_1, X_2, \dots, X_p\}$ ，当前的重要变量集 $\mathbf{r}_0 = \mathbf{F}$ 。进行第 p 次如下迭代过程：
 - 基于包含 $|\mathbf{S}|$ 个输入变量的集合 $\mathbf{S}$ ，训练预测模型 $M_{\mathbf{S}}$
 - 计算变量重要性： $w_i^{(M_{\mathbf{S}})}$ ，以 $[w_i^{(M_{\mathbf{S}})}]^2$ 作为重要性的度量
 - 找到 $\min([w_i^{(M_{\mathbf{S}})}]^2)$ 的变量 X_i ，将其从 $\mathbf{S}$ 中剔除，且 $\mathbf{r}_p \leftarrow \mathbf{S}$ 。 $\mathbf{r}_p$ 为第 p 次迭代后的重要变量子集
 - 重复上述步骤，直到 $\mathbf{S} = \{\}$ 迭代结束
 - 迭代结束将得到若干个重要变量子集： $\mathbf{r}_p \subset \cdots \mathbf{r}_2 \subset \mathbf{r}_1 \subset \mathbf{r}_0$

- 递归式特征剔除法示例：
 - 预测模型分别为Logistic回归和支持向量机
 - 找到重要性Top 80的变量 Chapter10-3.ipynb

```

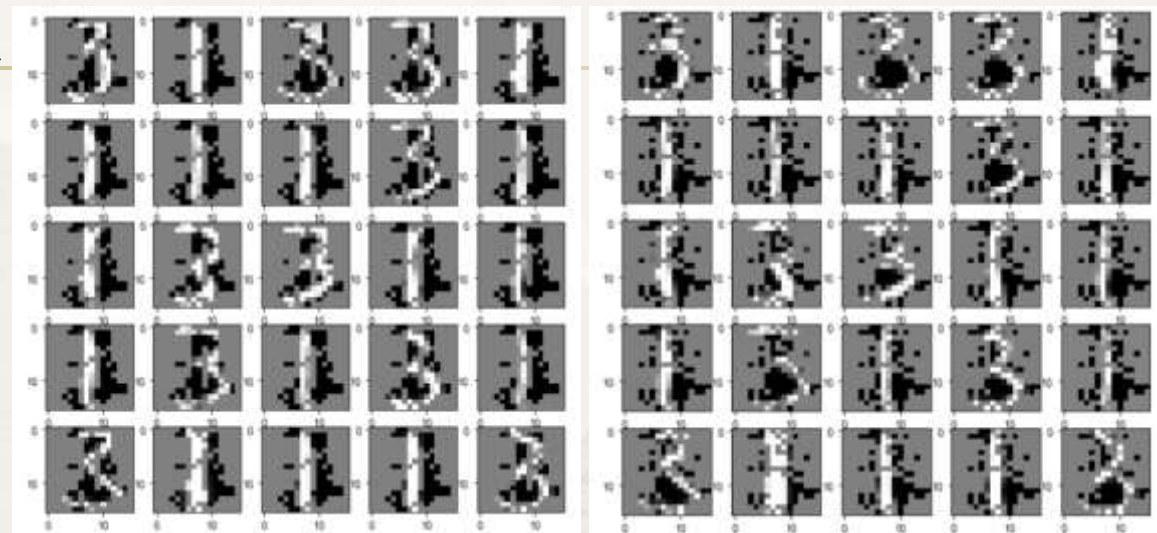
1 data = pd.read_table('邮政编码数据.txt', sep=' ', header=None)
2 tmp = data.loc[(data[0]==1) | (data[0]==3)]
3 X = tmp.iloc[:, 1:-1]
4 Y = tmp.iloc[:, 0].astype(str)
5 np.random.seed(1)
6 ids = np.random.choice(len(X), 25)
7 estimators = [LM.LogisticRegression(), svm.SVC(kernel='linear')]
8 for estimator in estimators:
9     selector = RFE(estimator=estimator, n_features_to_select=80)
10    selector.fit(X, Y)
11    print('变量重要性排名 %s'%selector.ranking_[0:5])
12    Xtmp = selector.inverse_transform(selector.transform(X))
13    plt.figure(figsize=(5, 5), dpi=150)
14    for i, item in enumerate(ids):
15        img = np.array(Xtmp[item, :]).reshape((16, 16))
16        plt.subplot(5, 5, i+1)
17        plt.imshow(img, cmap=cm.gray)
18    plt.show()
19

```

变量重要性排名 [176 170 130 69 27]

变量重要性排名 [177 176 110 69 7]

查看筛选的情况： `print(selector.support_)` True/False
`print(selector.get_support(True))`



- 降维处理的其他策略：空间变换

